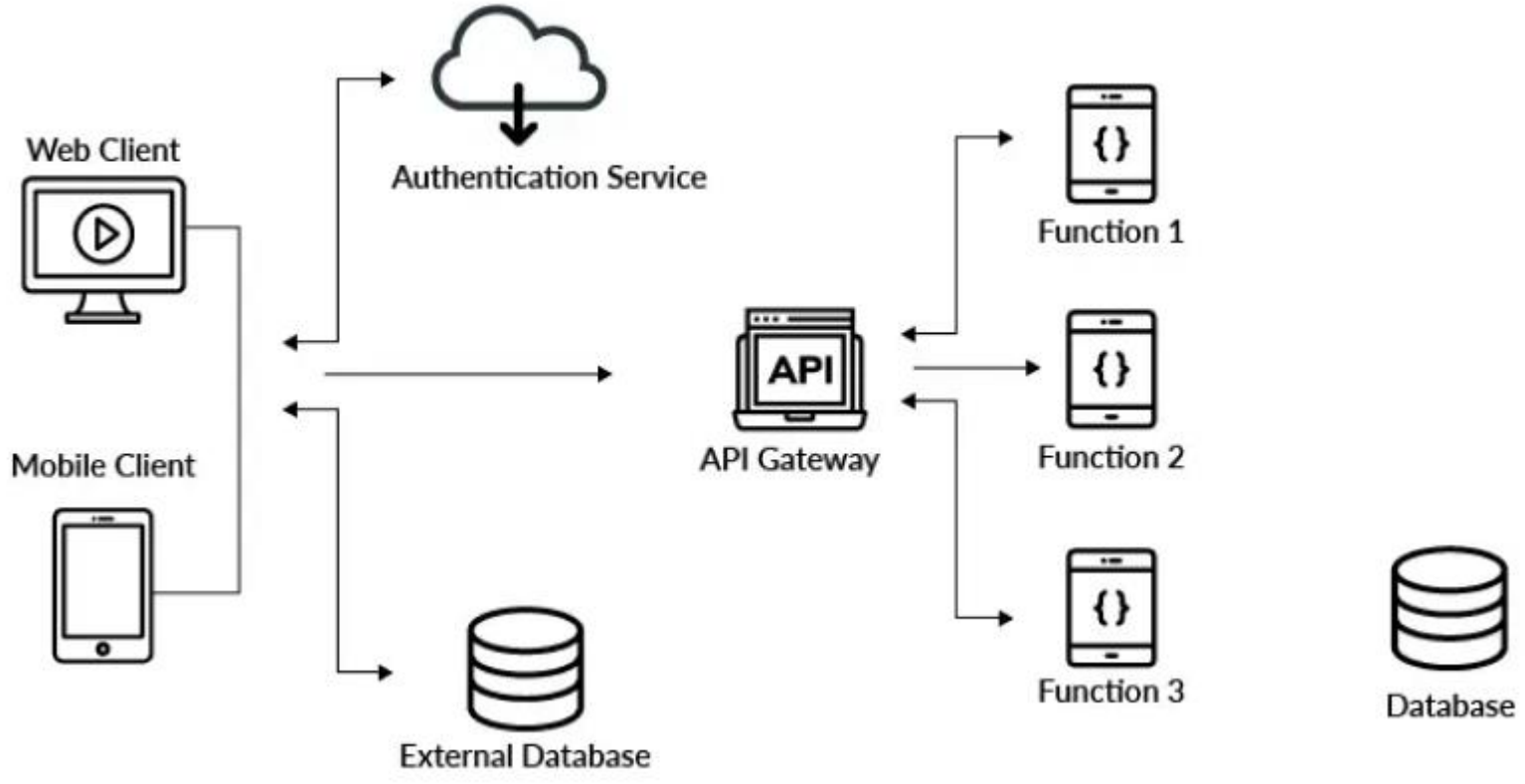
Software Architecture

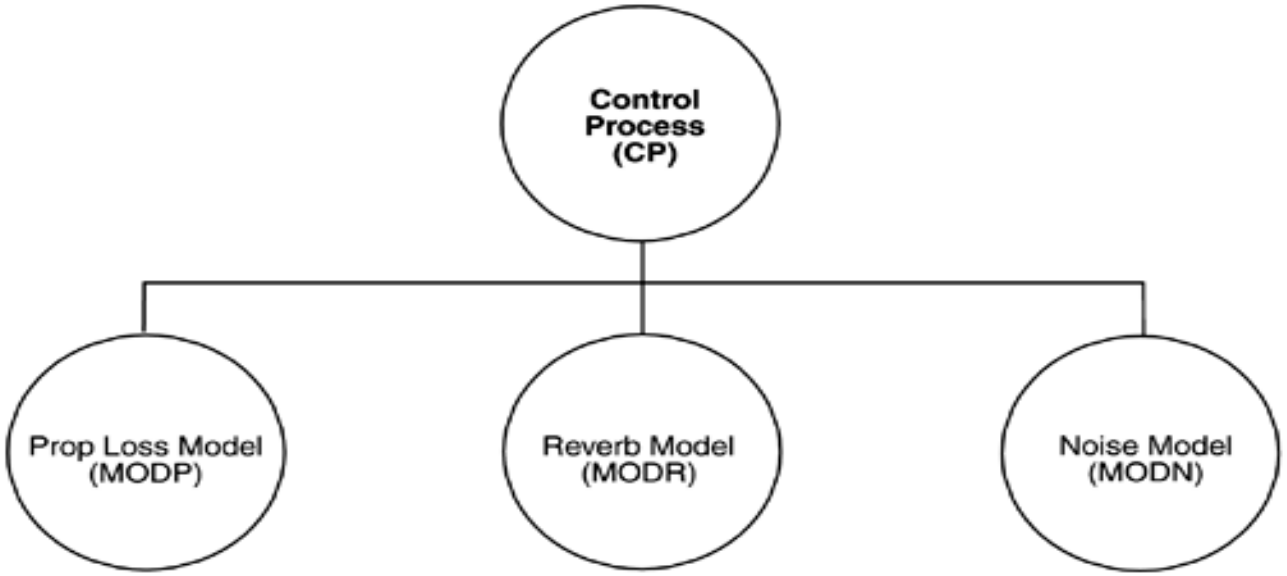
Session 2



Today We Explore

- ✅ What software architecture really means
- ✅ How systems are structured
- ✅ Architecture patterns
- ✅ Why architecture matters

What Software Architecture Is and What It Isn't?



- What can we tell from the figure?

What Software Architecture Is and What It Isn’t?

What Can We Guess?

- ✅ components exist
- ✅ systems connected
- ✅ relationships exist

What Software Architecture Is and What It Isn't?

- Most primitive definition of a Software architecture

" SW Architecture is a set of components and connections among them"

- If we accept this primitive definition, we can tell this is **SW architecture**.

- But, what we can not tell from the diagram?

What we can not tell from the diagram?

Discussion: “What information is missing?”

- responsibilities
- communication type
- timing
- behavior

What we can not tell from the diagram?

What are the responsibilities of the elements?

- What they does?
- What is their function in the system?

What we can not tell from the diagram?

What is the significance of the connections?

- Do the connections mean that the elements communicate with each other?
- Control each other
- Send data to each other
- Use each other
- Invoke each other
- Synchronize with each other
- Share some information or hiding secret with each other
- What are the mechanisms for the communication?

What we can not tell from the diagram?

What is the significance of the layout?

- Why is CP on a separate level?
- Does it call the other three elements, and are the others not allowed to call it?
- Does it contain the other three in an implementation unit sense?

What is Software Architecture?

Before take a decision, we must raise these questions

- what the elements are?
- how they cooperate to accomplish the purpose of the system?

The diagram does not show a software architecture

What is Software Architecture?

"The software architecture of a program or computing system is the ***structure*** or ***structures*** of the system, which comprise ***software elements***, the ***externally visible properties*** of those elements, and the ***relationships*** among them."

"***Externally visible***" properties are those assumptions other elements can make of an element, such as its provided services, shared resource usage.

Software Architecture in Practice
Len Bass, Paul Clements, and Rick Kazman

What is Software Architecture?

- Example: Online Food Delivery Platform (Ex: Uber Eats)
- Structure of the System
  - Logical Structure- Divided into three main layers
    - Frontend Layer: User-facing applications for customers, delivery personnel, and restaurant staff.
    - Business Logic Layer: Processes orders, handles payment processing, and manages restaurant and delivery assignments.
    - Data Layer: Stores user profiles, orders, payment records, and delivery history.
    - Each layer interacts but maintains separation of concerns.
  - Physical Structure:
    - Microservices architecture with services deployed in cloud containers, connected via APIs and message brokers.

What is Software Architecture?

- Example: Online Food Delivery Platform (Ex: Uber Eats)
- Software Elements - Examples of elements
  - A User Service handles user authentication and profiles.
  - An Order Management Service manages orders from creation to completion.
  - A Delivery Assignment Service assigns delivery personnel based on location and availability.
  - A Database stores data in relational or NoSQL format.

What is Software Architecture?

- Example: Online Food Delivery Platform (Ex: Uber Eats)
- Externally Visible Properties
  - User Service- Provides an API for user login, signup, and profile retrieval.
    Visible externally as a REST API endpoint (e.g., /api/v1/login).
  - Order Management Service- Accepts new orders via an API endpoint and updates their status.
    Supported operations (create order, cancel order).Expected response times.
    Communication protocol (e.g., HTTPS).
  - Delivery Assignment Service- Provides real-time location updates for delivery personnel.
    Visible to the frontend app through WebSocket or push notifications.

What is Software Architecture?

- Example: Online Food Delivery Platform (Ex: Uber Eats)
- Relationships Among Them
  - Order Service and Delivery Service- After an order is created, the Order Service communicates with the Delivery Service to assign a delivery partner.

    Communication might occur via a message broker like Kafka or RabbitMQ.
  - Frontend and Backend- The customer app (frontend) calls the Order Management API to place an order and receives a real-time update via WebSocket about delivery progress.
  - Data Relationships - The Order Service stores order details in the database, which is queried later by the Delivery Service to track order status.

What is Software Architecture?

- Architecture defines software elements.

- The architecture is principal abstraction (sketchy summary) of a system to be built, that hold back details of elements and how they use or interact with other elements.

What is Software Architecture?

- The architecture can comprise **more than one kind of structure.**

- The composition (structure) of the system can be described in different ways (functionality, the way the elements interact, memory usage….)

What is Software Architecture?

Example 1: SW projects can be divided accordingly to functions its carry out. These units can be assigned to implementation teams. **(This is one kind of structure often used to describe a system.)**

Example 2: The way the elements interact with each other at runtime to carry out the system's function.

All these structures are the software architecture.

What is Software Architecture?

- Every computing system with software has a **software architecture** because every system can be shown to comprise elements and the relations among them.

- Even though every system has an architecture, it does not necessarily that the architecture is known to anyone.

  - (People who designed the system are gone, documentation has vanished (or was never produced), source code has been lost (or was never delivered), and all we have is the executing binary code.)

What is Software Architecture?

- The **behaviour (relationship)** of each element is part of the architecture.

- Elements interact with other elements through behaviors, which is clearly part of the architecture.

- In ***OOP*** behavior exposes through ***methods*** (functions in some programming languages).

Architectural Patterns, Reference Models, and Reference Architectures

There are **three important** architectural structures

- Architectural pattern
- Reference model
- Reference architecture

Architectural Patterns

- Patterns represent known solutions to some problems (Ready made solution to a problem).
- An **architectural pattern** is a description of software elements and relation types together with a set of constraints on how they may be used.
- It is a description or template for how to solve a problem, not the complete solution, that can be used in many different situations.

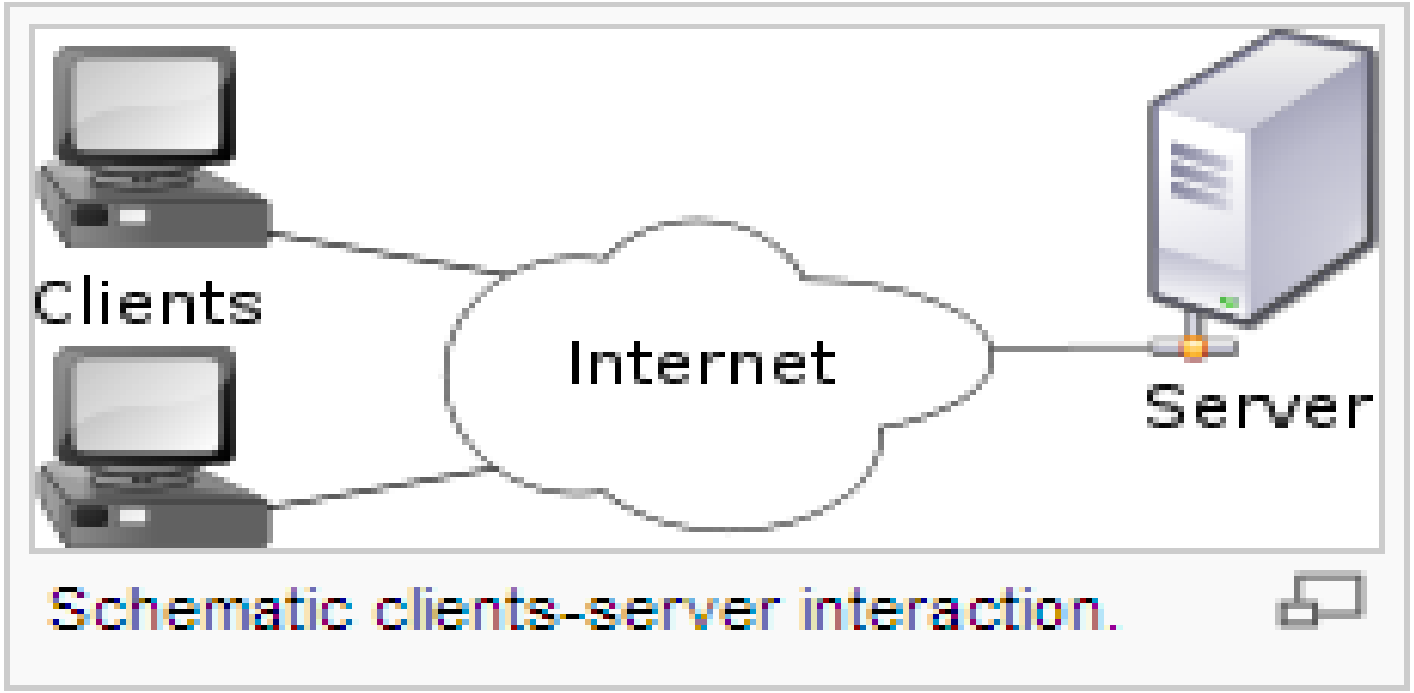
Architectural Patterns

- For example, **client-server** is a common architectural pattern used for communication.
- Client and server are two **element types**, and their coordination is described in terms of the **protocol** that the server uses to communicate with each of its clients.
- An architectural pattern is not an architecture, but it still conveys a useful image of the system.

Architectural Patterns

Object-oriented design patterns typically show ***relationships*** and interactions between ***classes*** or ***objects***, without specifying the final application classes or objects that are involved.

Client-server Patterns



Schematic clients-server interaction.

The server provides functions or services to one or many clients.

Protocols used: HTTP, SMTP, Telnet, and DNS.

Client-server Patterns

Example 1:

Email exchange, web access and database access are built on the client–server model.

Users accessing banking services from their computer, use a web browser client to send a request to a web server at a bank office, which in turn satisfies client request by sending required information.

Architectural Patterns

- More Examples:
  - **Layered Architecture**
    - Separate responsibilities in a large, complex system to ensure maintainability and scalability. (Presentation Layer, Business Logic Layer, Data Access Layer)
  - **Microservices Architecture**
    - Break the system into small, independent services that communicate via APIs.
    - Ex: Large systems like Amazon
  - **Event-Driven Architecture**
    - Use an event-driven model where components communicate by producing and consuming events.
    - Ex: IoT Applications

Architectural Patterns

- Advantage

  Patterns exhibit known quality attributes. This is why the architect chooses a particular pattern and not one at random.

  Some patterns represent known solutions to performance problems, others lend themselves well to high-security systems, still others have been used successfully in high-availability systems.

- Choosing an architectural pattern is often the architect's first major design choice.

Reference model

- A **reference model** is a division of **functionality of the system** together with **data flow** between the pieces.
- A **reference model** is a standard **decomposition** of a known problem into **sub- problems** that cooperatively solve the problem.

- Examples:
  - OSI Model - Network communication systems
  - Model-View-Controller - Software design for user interfaces
  - Cloud service architecture – Cloud computing reference model

Reference model

- **Example**: Can you name the standard parts of a **database management system**? Can you explain in how the parts work together to accomplish their purpose?

  If so, it is because you have been taught a reference model of these applications.

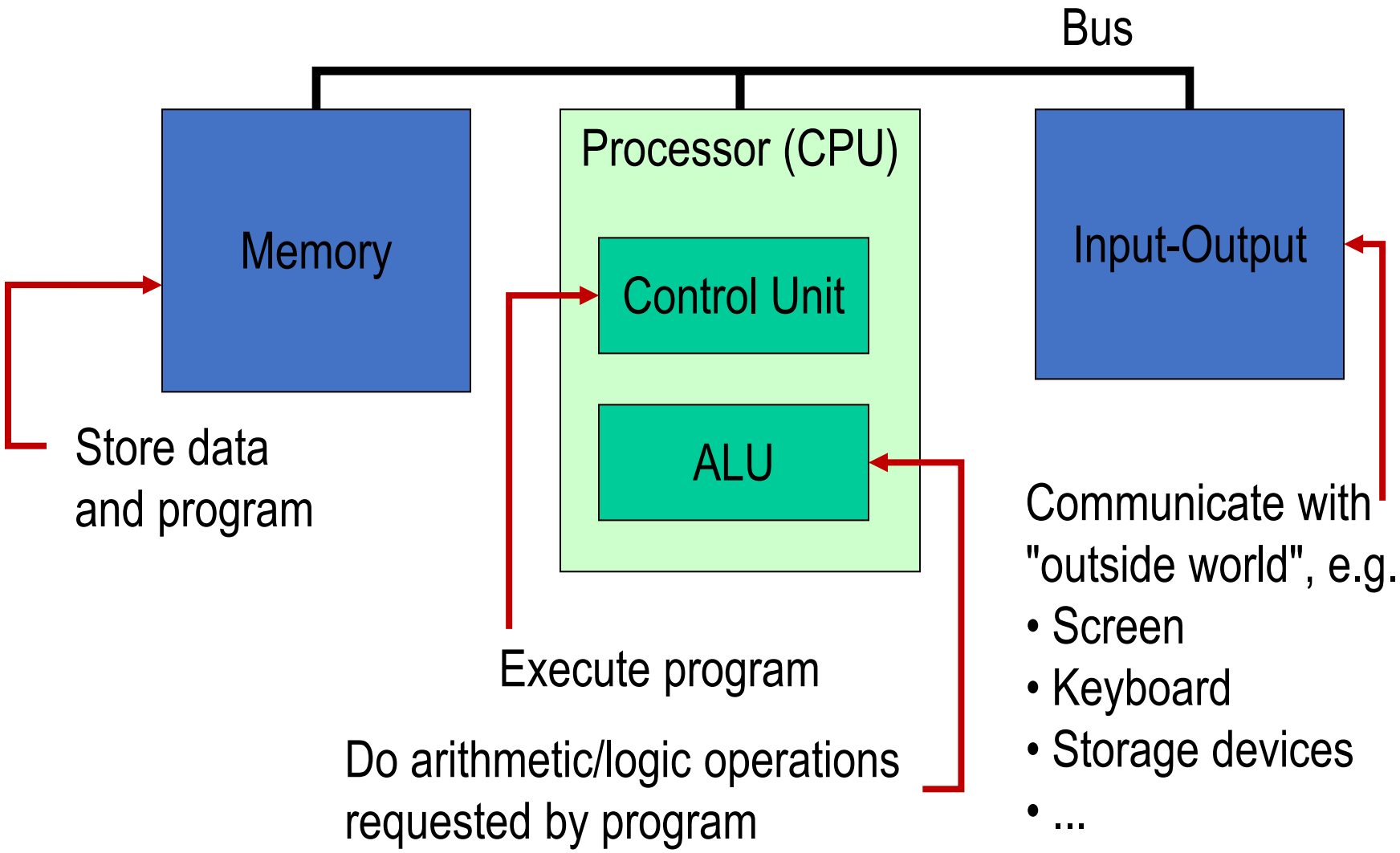
The Von Neumann Architecture

- Model for designing and building computers, based on the following three ***characteristics***:

  The computer consists of four main sub-systems:

  - Memory
  - ALU (Arithmetic/Logic Unit)
  - Control Unit
  - Input/Output System (I/O)

The Von Neumann Architecture



Reference architecture

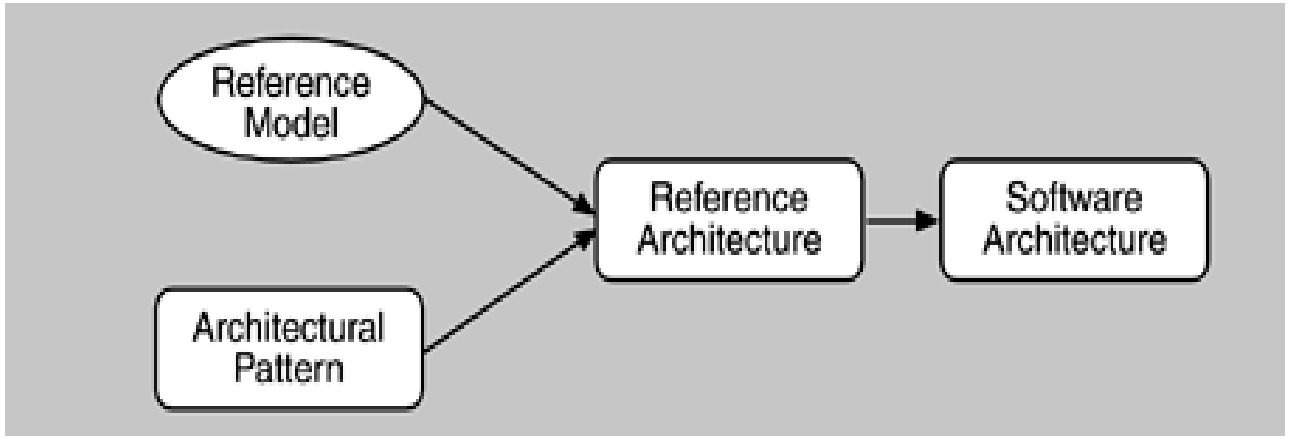
- A **reference architecture** is a reference model mapped onto software elements and the data flows between them.

- Reference model divides the functionality, a reference architecture is the mapping of that functionality onto a system components.

- A software element may implement part of a function or several functions.

Architectural Patterns, Reference Models, and Reference Architectures

Reference models, architectural patterns, and reference architectures are **not architectures**; they are useful concepts that capture elements of an architecture.



Why Is Software Architecture Important?

There are fundamentally three reasons for software architecture's importance:

- Communication among stakeholders.
- Early design decisions.
- Transferable abstraction (high level description) of a system

More to Read

Architecture is the vehicle for stakeholders communication

Each stakeholder of a software system is **concerned with different system characteristics** that are affected by the architecture.

Example:

- **User** is concerned that the system is **reliable** and **available** when needed.
- **Customer** is concerned that the architecture can be implemented on **schedule** and to **budget**.

Architecture is the vehicle for stakeholders communication

- Manager is worried **cost**, **schedule**, that the architecture will allow **teams to work largely independently** an so on.

- The architect is worried about strategies to **achieve all of those goals.**

Architecture is the vehicle for stakeholders communication

Software architecture represents a common **abstraction** of a system that system's stakeholders can use as a basis for mutual understanding (of the system), **negotiation**, **compromise**, and **communication**.

- Without such a language, it is difficult to understand large systems sufficiently to make the **early decisions** that influence both **quality** and **usefulness**.

Architecture represents the earliest set of design decisions

- Software architecture represent **earliest design decisions** about a system.
- These design decisions long last through system **development**, its **deployment**, and its **maintenance life**.
- These early decisions are the **most difficult to get correct** later in the development process, and they have the **most far-reaching effects**.

Architecture represents the earliest set of design decisions

These **early decisions** are the **most difficult to get correct** in the development process, and they have the **most far-reaching effects**.

1. The Architecture Defines **Constraints on Implementation**
2. The Architecture **Dictates Organizational Structure**
3. The Architecture **Enables a System's Quality Attributes**
4. Predicting System Qualities **by Studying the Architecture**

Architecture represents the earliest set of design decisions

These early decisions are the **most difficult to get correct** and the **hardest to change later** in the development process, and they have the **most far-reaching effects**.

5. The Architecture Makes It Easier to **Reason about and Manage Change**
6. The Architecture Helps in **Evolutionary Prototyping**
7. The Architecture Enables **More Accurate Cost and Schedule Estimates**

Architecture represents the earliest set of design decisions

1. The Architecture Defines **Constraints on Implementation**

Architecture sets the rules and boundaries within which the system must be built. It limits the technologies, methods, or design patterns developers can use.

Example:- If the architecture decides on using microservices, then developers can't build the system as a monolith. They must split features into independent services, use APIs for communication, and manage inter-service data flow.

Architecture represents the earliest set of design decisions

2. The Architecture Dictates **Organizational Structure**

The way software is structured often mirrors how the team is organized. This is known as Conway's Law.

Example:- If the system is architected into three main layers frontend, backend, and database, then the company may form three teams: UI team, API team, and Data team.

Architecture represents the earliest set of design decisions

3. The Architecture **Enables a System's Quality Attributes**.

Architecture impacts how well the system meets non-functional requirements like **performance, scalability, security, or reliability**.
Example:- To achieve high availability, the architecture may include load balancing and replication. This design choice enables the system to continue working even if one server fails.

Architecture represents the earliest set of design decisions

4. Predicting System Qualities by **Studying the Architecture**

   You can make educated guesses about how the system will behave (e.g., its **scalability or fault tolerance)** just by looking at its architecture.
   Example:- If an architecture uses a single database shared by all services, it may indicate a bottleneck under high load, potentially affecting scalability.

Architecture represents the earliest set of design decisions

5. The Architecture Makes It **Easier to Reason about Changes and Manage Changes to the system.**

A well-defined architecture helps you understand the impact of changes and manage them with less risk.

Example:- In a modular architecture, changing the payment module doesn’t affect the user authentication module. This isolation simplifies debugging and updates.

Architecture represents the earliest set of design decisions

6. The Architecture Helps in **Evolutionary Prototyping**.

Prototypes can be built step-by-step on top of a stable architecture. You can test ideas early without breaking the overall structure.
Example:- If you're building a shopping app, the architecture may first support browsing products. Later, you can add checkout and payment features without restructuring the system.

Architecture represents the earliest set of design decisions

7. The Architecture **Enables More Accurate Cost and Schedule Estimates**.

   Once the architecture is defined, you can better understand the complexity and time required for each component, leading to realistic planning.

   Example:- Knowing that the system uses third-party payment gateways, and that they require integration and compliance work, lets the project manager allocate time and budget for that specific part.

Why Is Software Architecture Important?

- Provides a Blueprint for Development
- Supports Scalability and Performance
- Enables Maintainability and Flexibility
- Improves Risk Management
- Aligns Technical Decisions with Business Goals
- Facilitates Communication
- Guides Technology Choices
- Helps with Estimation and Planning